

Dijkstra's Algorithm

#include <stdio.h>

```
int minDist (int dist[], int vis[], int n)
```

```
{
    int min = 10000, min-idx;
```

```
    for (int i = 0; i < n; i++)
```

```
        if (vis[i] == 0 && dist[i] == min)
```

```
            min = dist[i];
            min-idx = i;
```

```
    return min-idx;
```

```
void dijkstra (int graph[][], int src)
```

```
{
    int dist[100], vis[100];
```

```
    for (int i = 0; i < n; i++)
```

```
        dist[i] = 10000;
        vis[i] = 0;
```

```
}
```

```
    dist[src] = 0;
```

```
    for (int count = 0; count < n-1; count++)
        int u = minDist (dist, vis, n);
        vis[u] = 1;
```

```
for (int v=0; v<n; v++)
```

```
{
    if (!vis[v] && graph[u][v])
```

```
{
    if (dist[u] != 1000 &&
```

```
dist[u] + graph[u][v] < dist[v])
```

```
{
```

```
dist[v] = dist[u] + graph[u][v]
```

```
}
```

```
}
```

```
printf ("Vertex %d distance from  
source %d\n",
```

```
for (int i=0; i<n; i++)
```

```
printf ("%d\t\t%d\n",  
i, dist[i]);
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
int graph[100][100], n;
```

```
printf ("Enter the no. of nodes in graph:
```

```
scanf ("%d", &n);
```

```
printf ("Enter weight matrix: \n");
```

```
for (int i=0; i<n; i++)
```

```
{
```

```
for (int j=0; j<n; j++)
```

```
{
```

```
scanf ("%d", &graph[i][j]);
```

```
}
```

```
}
```

```
int source = 0;
dijkstra (graph, source);
return 0;
```

7

Output:

```
int source = 0;
clock_t start, end;
start = clock();
dijkstra (graph, source);
end = clock();
```

```
double time_taken =
(double) (end - start) / clock_per_sec;
```

```
printf ("Execution time: %f,
time - taken);
```

```
return 0;
```

8

Output:

Enter number of nodes: 5.

Enter weight matrix:

```
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
```

29/04/26

Vertex	Distance from source
0	0
1	10
2	50
3	30
4	60